

title: Guía de Implementación - Código TypeScript Completo **subtitle:** Manual de Integración del Sistema de Autenticación JWT **author:** Equipo de Desarrollo Frontend **date:** 15 de octubre de 2025 **version:** 1.0 **status:** Listo para Producción

Guía de Implementación TypeScript

Información del Documento

- **Código Base:** `auth_jwt_complete_implementation.ts`
- **Código XP (futuro):** `xp_level_complete_implementation.ts`
- **Estado Auth:**  Funcional y probado
- **Estado XP:**  Comentado para activación futura

Estructura de Archivos

Autenticación JWT (Actual)

```
src/
├── api/
│   ├── client.ts          # Cliente Axios configurado
│   └── services/
│       ├── auth.service.ts # Servicios de autenticación
│       └── types/
│           └── auth.types.ts # Tipos TypeScript
├── contexts/
│   └── AuthContext.tsx     # Contexto React de auth
├── hooks/
│   ├── useAuth.ts         # Hook principal de auth
│   ├── useRole.ts         # Hook de roles y permisos
│   └── useInactivityLogout.ts # Timeout de inactividad
├── components/
│   ├── Auth/
│   │   └── AuthForm.tsx    # Formulario login/register
│   ├── RoleGuard/
│   │   └── RoleGuard.tsx   # Guards de protección
│   └── RoleBadge/
│       └── RoleBadge.tsx   # Badge visual de rol
```

```
└─ config/
  └─ auth.config.ts    # Configuración central
```

Sistema XP (Futuro)

```
src/
├─ api/
│  └─ services/
│     └─ userLevel.service.ts # Servicios XP/Niveles
│  └─ types/
│     └─ xp.types.ts         # Tipos XP
│
├─ hooks/
│  └─ useUserLevel.ts        # Hook nivel usuario
│  └─ useLeaderboard.ts      # Hook leaderboard
│
├─ components/
│  └─ UserLevel/
│     ├── UserLevelDisplay.tsx # Display nivel/XP
│     └─ UserLevelDisplay.css
│  └─ Leaderboard/
│     ├── Leaderboard.tsx      # Tabla ranking
│     └─ Leaderboard.css
│  └─ XpHistory/
│     ├── XpHistory.tsx        # Historial XP
│     └─ XpHistory.css
│  └─ Admin/
│     └─ AwardXpPanel.tsx      # Panel admin XP
│
└─ pages/
   └─ LeaderboardPage.tsx      # Página completa
```

Instalación de Dependencias

```
bash
```

React Query (para manejo de datos async)

`npm install @tanstack/react-query @tanstack/react-query-devtools`

Axios (HTTP client)

`npm install axios`

Date utilities (para formateo fechas)

`npm install date-fns`

TypeScript types

`npm install --save-dev @types/react @types/node`

Configuración Inicial

1. Variables de Entorno

Crear archivo `.env.local`:

`bash`

URL base de la API

`REACT_APP_API_BASE_URL=http://localhost:8000/api/v1`

Opcional: Configurar timeouts

`REACT_APP_INACTIVITY_TIMEOUT=1800000 # 30 minutos`

`REACT_APP_API_TIMEOUT=10000 # 10 segundos`

2. Configuración de React Query

`typescript`

```
// src/App.tsx o src/index.tsx
```

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { ReactQueryDevtools } from '@tanstack/react-query-devtools';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60 * 1000, // 1 minuto
      retry: 2,
      refetchOnWindowFocus: false,
    },
  },
});

function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <AuthProvider>
        <YourApp />
      </AuthProvider>
      <ReactQueryDevtools initialIsOpen={false} />
    </QueryClientProvider>
  );
}
```

Implementación Paso a Paso

Paso 1: Configuración (`config/auth.config.ts`)

```
typescript
```

```
export const AUTH_CONFIG = {
  API_BASE_URL: process.env.REACT_APP_API_BASE_URL ||
    'http://localhost:8000/api/v1',

  AUTH_TYPE: 'jwt-bearer',

  ENDPOINTS: {
    login: '/login',
    register: '/register',
    logout: '/logout',
    me: '/me',
  },

  TOKEN: {
    STORAGE_KEY: 'auth_token',
    DURATION_DAYS: 15,
    HEADER_PREFIX: 'Bearer',
  },

  FEATURES: {
    hasRefreshToken: false,
    hasServerSideLogout: true,
    hasRoleSystem: true,
  },

  SECURITY: {
    INACTIVITY_TIMEOUT: parseInt(
      process.env.REACT_APP_INACTIVITY_TIMEOUT || '1800000'
    ),
    API_TIMEOUT: parseInt(
      process.env.REACT_APP_API_TIMEOUT || '10000'
    ),
    MAX_LOGIN_ATTEMPTS: 3,
  },
} as const;

export const validateAuthConfig = (): boolean => {
  if (!AUTH_CONFIG.API_BASE_URL) {
    console.error('✗ REACT_APP_API_BASE_URL not configured');
    return false;
  }
  console.log('✅ Auth configuration valid');
  return true;
};
```

Paso 2: Tipos (`api/types/auth.types.ts`)

typescript

```
export type UserRole = 'User' | 'Moderator' | 'Admin';
```

```
export interface User {  
  id: number;  
  name: string;  
  email: string;  
  role: UserRole;  
  email_verified_at: string | null;  
  created_at: string;  
  updated_at: string;  
}
```

```
export interface LoginRequest {  
  email: string;  
  password: string;  
}
```

```
export interface RegisterRequest {  
  name: string;  
  email: string;  
  password: string;  
  password_confirmation: string;  
}
```

```
export interface AuthResponse {  
  access_token: string;  
  token_type: 'bearer';  
  user: User;  
}
```

```
export interface LogoutResponse {  
  message: string;  
}
```

```
export interface ApiError {  
  message: string;  
  status?: number;  
  errors?: Record<string, string[]>;  
}
```

Paso 3: Cliente Axios (`api/client.ts`)

typescript

```
import axios, { AxiosInstance, AxiosError } from 'axios';
import { AUTH_CONFIG } from '@config/auth.config';

export const apiClient: AxiosInstance = axios.create({
  baseURL: AUTH_CONFIG.API_BASE_URL,
  timeout: AUTH_CONFIG.SECURITY.API_TIMEOUT,
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
  },
});

// Token management
export const setAuthToken = (token: string): void => {
  localStorage.setItem(AUTH_CONFIG.TOKEN.STORAGE_KEY, token);
  apiClient.defaults.headers.common['Authorization'] =
    `${AUTH_CONFIG.TOKEN.HEADER_PREFIX} ${token}`;
};

export const getAuthToken = (): string | null => {
  return localStorage.getItem(AUTH_CONFIG.TOKEN.STORAGE_KEY);
};

export const removeAuthToken = (): void => {
  localStorage.removeItem(AUTH_CONFIG.TOKEN.STORAGE_KEY);
  delete apiClient.defaults.headers.common['Authorization'];
};

// Initialize token from storage
const savedToken = getAuthToken();
if (savedToken) {
  apiClient.defaults.headers.common['Authorization'] =
    `${AUTH_CONFIG.TOKEN.HEADER_PREFIX} ${savedToken}`;
}

// Request interceptor
apiClient.interceptors.request.use(
  (config) => {
    const token = getAuthToken();
    if (token && config.headers) {
      config.headers['Authorization'] =
        `${AUTH_CONFIG.TOKEN.HEADER_PREFIX} ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
```

```
);
```

```
// Response interceptor
```

```
apiClient.interceptors.response.use(  
  (response) => response,  
  (error: AxiosError) => {  
    if (error.response?.status === 401) {  
      removeAuthToken();  
  
      const currentPath = window.location.pathname;  
      if (currentPath !== '/login') {  
        localStorage.setItem('redirect_after_login', currentPath);  
      }  
  
      window.location.href = '/login?reason=expired';  
    }  
  
    return Promise.reject(transformError(error));  
  }  
);
```

```
// Error transformer
```

```
const transformError = (error: AxiosError): ApiError => {  
  if (error.response?.data) {  
    const data = error.response.data as any;  
    return {  
      message: data.message || 'Error en la solicitud',  
      status: error.response.status,  
      errors: data.errors,  
    };  
  }  
  
  if (error.request) {  
    return {  
      message: 'Error de red. Verifica tu conexión.',  
      status: 0,  
    };  
  }  
  
  return {  
    message: error.message || 'Error desconocido',  
  };  
};
```

```
// Generic API call helper
```

```
interface ApiCallConfig {  
  method: 'GET' | 'POST' | 'PUT' | 'DELETE' | 'PATCH';
```

```
url: string;
data?: any;
params?: any;
}

export const apiCall = async <T>(config: ApiCallConfig): Promise<T> => {
  const response = await apiClient.request<T>({
    method: config.method,
    url: config.url,
    data: config.data,
    params: config.params,
  });

  return response.data;
};
```

Paso 4: Servicios (api/services/auth.service.ts)

typescript

```
import { apiCall } from '../client';
import { setAuthToken } from '../client';
import {
  LoginRequest,
  RegisterRequest,
  AuthResponse,
  LogoutResponse,
  User,
} from '../types/auth.types';
import { AUTH_CONFIG } from '@config/auth.config';

export const authService = {
  login: async (credentials: LoginRequest): Promise<AuthResponse> => {
    const response = await apiCall<AuthResponse>({
      method: 'POST',
      url: AUTH_CONFIG.ENDPOINTS.login,
      data: credentials,
    });

    setAuthToken(response.access_token);
    return response;
  },

  register: async (userData: RegisterRequest): Promise<AuthResponse> => {
    const response = await apiCall<AuthResponse>({
      method: 'POST',
      url: AUTH_CONFIG.ENDPOINTS.register,
      data: userData,
    });

    setAuthToken(response.access_token);
    return response;
  },

  logout: async (): Promise<LogoutResponse> => {
    try {
      const response = await apiCall<LogoutResponse>({
        method: 'POST',
        url: AUTH_CONFIG.ENDPOINTS.logout,
      });

      return response;
    } finally {
      removeAuthToken();
    }
  },
}
```

```
me: async (): Promise<User> => {  
  return apiCall<User>({  
    method: 'GET',  
    url: AUTH_CONFIG.ENDPOINTS.me,  
  });  
},  
};
```

Paso 5: Contexto (`contexts/AuthContext.tsx`)

typescript

```
import React, {
  createContext,
  useContext,
  useState,
  useEffect,
  ReactNode
} from 'react';
import { authService } from '@api/services/auth.service';
import { getAuthToken, removeAuthToken } from '@api/client';
import {
  User,
  LoginRequest,
  RegisterRequest
} from '@api/types/auth.types';

interface AuthContextValue {
  user: User | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  login: (credentials: LoginRequest) => Promise<void>;
  register: (userData: RegisterRequest) => Promise<void>;
  logout: () => Promise<void>;
  updateUser: (user: User) => void;
}

const AuthContext = createContext<AuthContextValue | undefined>(
  undefined
);

interface AuthProviderProps {
  children: ReactNode;
}

export const AuthProvider: React.FC<AuthProviderProps> = ({
  children
}) => {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    const initializeAuth = async () => {
      const token = getAuthToken();

      if (token) {
        try {
          const userData = await authService.me();

```

```
    setUser(userData);
  } catch (error) {
    console.error('Token validation failed:', error);
    removeAuthToken();
  }
}

setIsLoading(false);
};

initializeAuth();
}, []);
```

```
const login = async (credentials: LoginRequest): Promise<void> => {
  setIsLoading(true);
  try {
    const response = await authService.login(credentials);
    setUser(response.user);

    const redirectPath = localStorage.getItem('redirect_after_login');
    if (redirectPath) {
      localStorage.removeItem('redirect_after_login');
      window.location.href = redirectPath;
    }
  } finally {
    setIsLoading(false);
  }
};
```

```
const register = async (userData: RegisterRequest): Promise<void> => {
  setIsLoading(true);
  try {
    const response = await authService.register(userData);
    setUser(response.user);
  } finally {
    setIsLoading(false);
  }
};
```

```
const logout = async (): Promise<void> => {
  setIsLoading(true);
  try {
    await authService.logout();
  } finally {
    setUser(null);
    setIsLoading(false);
  }
}
```

```

};

const updateUser = (updatedUser: User) => {
  setUser(updatedUser);
};

return (
  <AuthContext.Provider
    value={{
      user,
      isAuthenticated: !!user,
      isLoading,
      login,
      register,
      logout,
      updateUser,
    }}
  >
    {children}
  </AuthContext.Provider>
);
};

export const useAuth = (): AuthContextValue => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within AuthProvider');
  }
  return context;
};

```

Paso 6: Sistema de Roles (`hooks/useRole.ts`)

typescript

```
import { useAuth } from '@contexts/AuthContext';
import { UserRole } from '@api/types/auth.types';

const roleHierarchy: Record<UserRole, number> = {
  'User': 1,
  'Moderator': 2,
  'Admin': 3,
};

interface RolePermissions {
  canCreateDisciplines: boolean;
  canEditOwnDisciplines: boolean;
  canEditAnyDiscipline: boolean;
  canViewUserStats: boolean;
  canManageUsers: boolean;
  canDeleteUsers: boolean;
  canViewAdminPanel: boolean;
}

export const useRole = () => {
  const { user, isAuthenticated } = useAuth();

  const hasMinimumRole = (requiredRole: UserRole): boolean => {
    if (!isAuthenticated || !user?.role) return false;
    return roleHierarchy[user.role] >= roleHierarchy[requiredRole];
  };

  const hasExactRole = (role: UserRole): boolean => {
    if (!isAuthenticated || !user?.role) return false;
    return user.role === role;
  };

  const permissions: RolePermissions = {
    canCreateDisciplines: hasMinimumRole('User'),
    canEditOwnDisciplines: hasExactRole('User'),
    canEditAnyDiscipline: hasMinimumRole('Moderator'),
    canViewUserStats: hasMinimumRole('Moderator'),
    canManageUsers: hasMinimumRole('Moderator'),
    canDeleteUsers: hasMinimumRole('Admin'),
    canViewAdminPanel: hasMinimumRole('Admin'),
  };

  return {
    user,
    userRole: user?.role || null,
    permissions,
  };
};
```

```
hasMinimumRole,  
hasExactRole,  
};  
};
```

Paso 7: Timeout de Inactividad (`hooks/useInactivityLogout.ts`)

typescript

```
import { useEffect, useCallback, useRef } from 'react';
import { useAuth } from '@contexts/AuthContext';
import { AUTH_CONFIG } from '@config/auth.config';

export const useInactivityLogout = (
  timeoutMs: number = AUTH_CONFIG.SECURITY.INACTIVITY_TIMEOUT
) => {
  const { isAuthenticated, logout } = useAuth();
  const timeoutRef = useRef<NodeJS.Timeout | null>(null);

  const resetTimer = useCallback(() => {
    if (timeoutRef.current) {
      clearTimeout(timeoutRef.current);
    }

    if (isAuthenticated) {
      timeoutRef.current = setTimeout(() => {
        console.warn('Session expired due to inactivity');
        logout();
        window.location.href = '/login?reason=inactivity';
      }, timeoutMs);
    }
  }, [isAuthenticated, logout, timeoutMs]);

  useEffect(() => {
    if (!isAuthenticated) return;

    const events = ['mousedown', 'keydown', 'scroll', 'touchstart', 'click'];

    events.forEach(event => {
      document.addEventListener(event, resetTimer, true);
    });

    resetTimer();

    return () => {
      if (timeoutRef.current) {
        clearTimeout(timeoutRef.current);
      }
      events.forEach(event => {
        document.removeEventListener(event, resetTimer, true);
      });
    };
  }, [isAuthenticated, resetTimer]);
};
```

Componentes UI

AuthForm (components/Auth/AuthForm.tsx)

typescript

```
import React, { useState } from 'react';
import { useAuth } from '@contexts/AuthContext';
import { ApiError } from '@api/types/auth.types';

interface AuthFormProps {
  mode?: 'login' | 'register';
  onSuccess?: () => void;
}

export const AuthForm: React.FC<AuthFormProps> = ({
  mode = 'login',
  onSuccess
}) => {
  const { login, register, isLoading } = useAuth();
  const [isLoginMode, setIsLoginMode] = useState(mode === 'login');
  const [error, setError] = useState<string | null>(null);
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    password: '',
    password_confirmation: '',
  });

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setFormData(prev => ({
      ...prev,
      [e.target.name]: e.target.value,
    }));
    setError(null);
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);

    try {
      if (isLoginMode) {
        await login({
          email: formData.email,
          password: formData.password,
        });
      } else {
        if (formData.password !== formData.password_confirmation) {
          setError('Las contraseñas no coinciden');
          return;
        }
      }
    }
  }
}
```

```

    await register(formData);
  }

  onSuccess?();
} catch (err: any) {
  const apiError = err as ApiError;

  if (apiError.errors) {
    const firstError = Object.values(apiError.errors)[0];
    setError(firstError[0]);
  } else {
    setError(apiError.message || 'Error de autenticación');
  }
}
};

return (
  <form onSubmit={handleSubmit} className="auth-form">
    {error && (
      <div className="error-message" role="alert">
        {error}
      </div>
    )}

    {!isLoginMode && (
      <div className="form-group">
        <label htmlFor="name">Nombre</label>
        <input
          type="text"
          id="name"
          name="name"
          value={formData.name}
          onChange={handleChange}
          required
          disabled={isLoading}
        />
      </div>
    )}

    <div className="form-group">
      <label htmlFor="email">Email</label>
      <input
        type="email"
        id="email"
        name="email"
        value={formData.email}

```

```
    onChange={handleChange}
    required
    disabled={isLoading}
  />
</div>

<div className="form-group">
  <label htmlFor="password">Contraseña</label>
  <input
    type="password"
    id="password"
    name="password"
    value={formData.password}
    onChange={handleChange}
    required
    minLength={8}
    disabled={isLoading}
  />
</div>

{!isLoadingMode && (
  <div className="form-group">
    <label htmlFor="password_confirmation">Confirmar Contraseña</label>
    <input
      type="password"
      id="password_confirmation"
      name="password_confirmation"
      value={formData.password_confirmation}
      onChange={handleChange}
      required
      minLength={8}
      disabled={isLoading}
    />
  </div>
)}

<button type="submit" disabled={isLoading} className="submit-button">
  {isLoading ? 'Procesando...' : (isLoadingMode ? 'Iniciar Sesión' : 'Registrarse')}
</button>

<button
  type="button"
  onClick={() => {
    setIsLoginMode(!isLoadingMode);
    setError(null);
  }}
  className="toggle-mode-button"
```

```
        disabled={isLoading}
      >
        {isLoadingMode ? '¿No tienes cuenta? Regístrate' : '¿Ya tienes cuenta? Inicia sesión'}
      </button>
    </form>
  );
};
```

RoleGuard (components/RoleGuard/RoleGuard.tsx)

typescript

```
import React from 'react';
import { useAuth } from '@/contexts/AuthContext';
import { useRole } from '@/hooks/useRole';
import { UserRole } from '@/api/types/auth.types';
```

```
interface RoleGuardProps {
  children: React.ReactNode;
  allowedRoles?: UserRole[];
  minimumRole?: UserRole;
  requireAuth?: boolean;
  fallback?: React.ReactNode;
}
```

```
export const RoleGuard: React.FC<RoleGuardProps> = ({
  children,
  allowedRoles,
  minimumRole,
  requireAuth = true,
  fallback = null,
}) => {
  const { isAuthenticated } = useAuth();
  const { user, hasMinimumRole } = useRole();

  if (requireAuth && !isAuthenticated) {
    return <{fallback}>;
  }

  if (allowedRoles && user?.role && !allowedRoles.includes(user.role)) {
    return <{fallback}>;
  }

  if (minimumRole && !hasMinimumRole(minimumRole)) {
    return <{fallback}>;
  }

  return <{children}>;
};
```

// Componentes especializados

```
export const AdminOnly: React.FC<{
  children: React.ReactNode;
  fallback?: React.ReactNode
}> = ({ children, fallback = null }) => (
  <RoleGuard allowedRoles={['Admin']} fallback={fallback}>
    {children}
  </RoleGuard>
)
```

```
);

export const ModeratorPlus: React.FC<{
  children: React.ReactNode;
  fallback?: React.ReactNode
}> = ({ children, fallback = null }) => (
  <RoleGuard minimumRole="Moderator" fallback={fallback}>
    {children}
  </RoleGuard>
);

export const AuthenticatedOnly: React.FC<{
  children: React.ReactNode;
  fallback?: React.ReactNode
}> = ({ children, fallback = null }) => (
  <RoleGuard requireAuth={true} fallback={fallback}>
    {children}
  </RoleGuard>
);
```

Integración en App.tsx

```
typescript
```

```
import React, { useEffect } from 'react';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { AuthProvider } from './contexts/AuthContext';
import { useAuth } from './contexts/AuthContext';
import { useInactivityLogout } from './hooks/useInactivityLogout';
import { validateAuthConfig } from './config/auth.config';
import { AuthForm } from './components/Auth/AuthForm';
import Dashboard from './pages/Dashboard';
```

```
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60 * 1000,
      retry: 2,
      refetchOnWindowFocus: false,
    },
  },
});
```

```
function App() {
  useEffect(() => {
    validateAuthConfig();
  }, []);

  return (
    <QueryClientProvider client={queryClient}>
      <AuthProvider>
        <AppContent />
      </AuthProvider>
    </QueryClientProvider>
  );
}
```

```
function AppContent() {
  const { isAuthenticated, isLoading } = useAuth();

  // Activar timeout de inactividad
  useInactivityLogout();

  if (isLoading) {
    return <div className="loading">Cargando...</div>;
  }

  return (
    <div className="app">
      {isAuthenticated ? <Dashboard /> : <AuthForm />}
    </div>
  );
}
```

```
</div>  
);  
}  
  
export default App;
```

Uso en Componentes

Ejemplo: Dashboard con Guards

typescript

```
import React from 'react';
import { useAuth } from '@/contexts/AuthContext';
import { useRole } from '@/hooks/useRole';
import {
  AdminOnly,
  ModeratorPlus,
  AuthenticatedOnly
} from '@/components/RoleGuard/RoleGuard';

export const Dashboard: React.FC = () => {
  const { user, logout } = useAuth();
  const { permissions } = useRole();

  const handleLogout = async () => {
    try {
      await logout();
      window.location.href = '/login';
    } catch (error) {
      console.error('Logout failed:', error);
    }
  };

  return (
    <div className="dashboard">
      <header>
        <h1>Bienvenido, {user?.name}</h1>
        <button onClick={handleLogout}>Cerrar Sesión</button>
      </header>

      <nav>
        <AuthenticatedOnly>
          <a href="/disciplines">Disciplinas</a>
          <a href="/communities">Comunidades</a>
        </AuthenticatedOnly>

        <ModeratorPlus>
          <a href="/moderation">Moderación</a>
          <a href="/users">Usuarios</a>
        </ModeratorPlus>

        <AdminOnly>
          <a href="/admin">Administración</a>
        </AdminOnly>
      </nav>

      <main>
```

```

    {permissions.canCreateDisciplines && (
      <button>Crear Disciplina</button>
    )}

    <ModeratorPlus>
      <section>
        <h2>Estadísticas</h2>
        { /* Contenido solo para Moderator+ */ }
      </section>
    </ModeratorPlus>
  </main>
</div>
);
};

```

Testing

Setup de Tests

```

typescript

// setupTests.ts

import '@testing-library/jest-dom';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

export const createTestQueryClient = () => new QueryClient({
  defaultOptions: {
    queries: {
      retry: false,
    },
    mutations: {
      retry: false,
    },
  },
});

export const wrapper = ({ children }: { children: React.ReactNode }) => (
  <QueryClientProvider client={createTestQueryClient()}>
    {children}
  </QueryClientProvider>
);

```

Test de Login

```

typescript

```

```
import { renderHook, waitFor } from '@testing-library/react';
import { useAuth } from '@contexts/AuthContext';
import { AuthProvider } from '@contexts/AuthContext';

describe('useAuth', () => {
  it('should login successfully', async () => {
    const wrapper = ({ children }: any) => (
      <AuthProvider>{children}</AuthProvider>
    );

    const { result } = renderHook(() => useAuth(), { wrapper });

    await act(async () => {
      await result.current.login({
        email: 'test@example.com',
        password: 'password123',
      });
    });

    expect(result.current.isAuthenticated).toBe(true);
    expect(result.current.user).toBeDefined();
  });
});
```

Activación del Sistema XP (Futuro)

Paso 1: Verificar Endpoints Backend

bash

Verificar que estos endpoints existan:

`curl http://localhost:8000/api/v1/users/1/level`

`curl http://localhost:8000/api/v1/leaderboard`

Paso 2: Descomentar Código

typescript

```
// components/Layout/Header.tsx

// ❌ ELIMINAR mock:
// const userLevel = 12;
// const userXp = 2450;

// ✅ DESCOMENTAR:
import { UserLevelDisplay } from '@components/UserLevel/UserLevelDisplay';

// En JSX:
<UserLevelDisplay size="small" />
```

Paso 3: Agregar Ruta

```
typescript

// App.tsx o Router.tsx

import { LeaderboardPage } from '@pages/LeaderboardPage';

<Route path="/leaderboard" element={
  <AuthenticatedOnly>
    <LeaderboardPage />
  </AuthenticatedOnly>
} />
```

Troubleshooting

Error: "CORS policy"

Solución: Configurar CORS en backend Laravel:

```
php

// config/cors.php
'paths' => ['api/*'],
'allowed_origins' => ['http://localhost:3000'],
'allowed_methods' => ['*'],
'allowed_headers' => ['*'],
```

Error: "401 Unauthorized"

Diagnóstico:

1. Verificar token en localStorage

2. Verificar header Authorization en network tab
3. Verificar que el token no haya expirado

Solución:

```
typescript

// Limpiar y relloguear
localStorage.clear();
window.location.href = '/login';
```

Error: "Network Error"

Diagnóstico:

1. Verificar que backend esté corriendo
2. Verificar URL en `.env.local`
3. Verificar firewall/antivirus

Checklist de Producción

- ☐ Variables de entorno configuradas
- ☐ HTTPS habilitado
- ☐ Timeout de inactividad probado
- ☐ Errores manejados correctamente
- ☐ Tests pasando al 100%
- ☐ CORS configurado
- ☐ Rate limiting verificado
- ☐ Logging configurado
- ☐ Performance optimizada
- ☐ Documentación actualizada

Referencias

- **Código completo:** Artefactos generados anteriormente
 - **React Query Docs:** <https://tanstack.com/query/latest>
 - **Axios Docs:** <https://axios-http.com/>
 - **TypeScript Handbook:** <https://www.typescriptlang.org/docs/>
-

Documento generado: 15 de octubre de 2025

Versión: 1.0

Estado:  Listo para implementación

Soporte: [Ver código fuente en artefactos](#)